

PROTOCOL.md — Spot Welder Serial/TCP Telemetry Protocol

Reverse-engineered descriptive reference — documents observed behavior as of the current firmware.

This is **not** a shared header file. Protocol definitions are independently implemented in:

- STM32G474CE/src/main.c (packet producers)
- ESP32P4/main/welder_main.cpp and wifi_bridge.cpp (relay + enrichment)
- ESP32_8048S043C/src/main.cpp (legacy firmware)
- Spot-Welder-Server/app.py (Flask consumer)

Any field name, type, or unit change requires coordinated updates across all four codebases.

Protocol Overview

Transport: ASCII lines over UART (576000 8N1) and TCP (:8888).

Format: Comma-separated `PACKET_TYPE,key=value,key=value,...` or positional CSV.

Component Roles

Component	Role
STM32G474CE	Sole producer of <code>STATUS</code> , <code>STATUS2</code> , <code>WELD_DONE</code> , and <code>WAVEFORM_*</code> packets. Real-time weld controller with ADC/shunt/INA226 telemetry.
ESP32-P4	Enriches <code>STATUS</code> by appending WiFi/system/energy fields. Relays <code>STATUS2</code> , <code>WAVEFORM_*</code> , and <code>WELD_DONE</code> packets raw (transparent passthrough). Produces <code>DISPLAY</code> packets at 1 Hz for UI smoothed voltages.
ESP32_8048S043C (legacy)	Produces <code>DISPLAY</code> and <code>CELLS</code> packets. Relays <code>WAVEFORM_*</code> packets. Not the primary firmware.
Flask Server	Consumes all packets. Re-parses telemetry for web dashboard.

Packet Catalog

STATUS — System State and Weld Parameters

Producer: STM32

Enricher: ESP32-P4 (appends WiFi/system/energy fields)

Consumer: Flask

Frequency: ~10 Hz during idle, faster during weld

STM32-produced fields:

Field	Type	Units	Description	Notes
mode	string	—	Control mode: JOULE , MANUAL , DUAL	
power	uint8	Percent	Weld power set-point, clamped 50-100	Not watts. Represents PWM duty cycle percentage.
pulse_ms	uint16	milliseconds	Weld pulse duration	
preheat_ms	uint16	milliseconds	Preheat phase duration	
gap_ms	uint16	milliseconds	Gap between preheat and main weld	
joule_target_j	float	joules	Target energy for joule mode	Primary field (see legacy note below).
joule_target	float	joules	Legacy/deprecated alias of joule_target_j	Emitted only in joule mode; redundant.
vcap	float	volts	Capacitor bank voltage	
temp	float	°C	Thermistor temperature	
ichg	float	amps	Charger current (from ADC shunt)	
chg_en	uint8	boolean	Charger MOSFET state (1=on, 0=off)	
state	string	—	Weld state machine: IDLE , WELD , DONE , etc.	

Field	Type	Units	Description	Notes
<code>fp</code>	uint8	boolean	Foot pedal state (1=pressed, 0=released)	

ESP32-P4 enrichment fields (appended after STM32 fields):

Field	Type	Units	Description
<code>rssi</code>	int8	dBm	WiFi signal strength
<code>heap</code>	uint32	bytes	Free heap memory
<code>uptime</code>	uint32	seconds	Firmware uptime
<code>weld_count</code>	uint32	—	Total weld count (persisted in NVS)
<code>energy_lifetime_j</code>	float	joules	Cumulative energy across all welds

Legacy note: `joule_target` is a redundant alias of `joule_target_j`, both populated from the same `joule_target_j` variable in `main.c:679-680`. Flask prefers `joule_target_j`.

STATUS2 — Battery Pack Telemetry

Producer: STM32 (via INA226 sensors)

Consumer: Flask

Relay: ESP32-P4 (raw passthrough, no enrichment)

Frequency: ~10 Hz

Field	Type	Units	Description	Notes
<code>ina_ok</code>	uint8	boolean	INA226 sensor health (1=ok, 0=fault)	
<code>chg_en</code>	uint8	boolean	Charger enable state	Duplicate of <code>STATUS.chg_en</code>
<code>vpack</code>	float	volts	Total pack voltage (sum of cells)	
<code>vlow</code>	float	volts	Lowest cell voltage	
<code>vmid</code>	float	volts	Middle cell voltage	
<code>cell1</code>	float	volts	Cell 1 voltage	
<code>cell2</code>	float	volts	Cell 2 voltage	
<code>cell3</code>	float	volts	Cell 3 voltage	Only 3 cells emitted.
<code>ichg</code>	float	amps	Charger current (from INA226)	

Cell count note: STM32 emits exactly **3 cells** (`cell1` , `cell2` , `cell3`). Flask contains scaffolding for `cell4` aliases (`app.py:1406-1415`), but since `cell4` is never emitted, this code path is a **silent no-op** (harmless legacy). Flask does not error or default; the alias simply remains unpopulated.

WELD_DONE — Weld Completion Event

Producer: STM32

Consumer: Flask

Relay: ESP32-P4 (raw passthrough)

Frequency: Once per weld completion

Emitted as `EVENT,WELD_DONE,key=value,...`

Field	Type	Units	Description	Notes
peak_current	float	amps	Peak current during weld	
peak_voltage	float	volts	Peak voltage during weld	
peak_power	float	watts	Peak instantaneous power	
avg_power	float	watts	Average power during pulse	
energy_weld_j	float	joules	Integrated weld energy (joule-integration method)	Primary field (see legacy note below).
energy_j	float	joules	Legacy/deprecated alias of energy_weld_j	Redundant; both populated from energy_weld_joules.
energy_cap_j	float	joules	Energy via cap-bank ΔV method: $0.5 * C * (V_{pre}^2 - V_{post}^2)$	Separate calculation method.
duration_us	uint32	microseconds	Actual weld pulse duration	
samples	uint16	—	Number of waveform samples captured	

Legacy note: energy_j and energy_weld_j are **redundant**; both are populated from the same energy_weld_joules variable (main.c:3189-3190). A stale comment at main.c:3157 suggests energy_j was once the cap-bank ΔV method, but that value now resides in energy_cap_j . Flask prefers energy_weld_j and uses energy_j as a fallback (app.py:2067-2070).

WAVEFORM_* — High-Speed Weld Capture Data

Producer: STM32 (sole producer)

Consumer: Flask

Relay: ESP32-P4 (**transparent raw relay** — does not parse, consume, or store waveform data)

Frequency: Burst after weld completion

The waveform data is sent as a chunked stream:

WAVEFORM_START

Signals the beginning of a waveform burst.

Format: `WAVEFORM_START,samples=N,preheat=I1,gap=I2,main=I3`

Field	Type	Units	Description	Notes
<code>samples</code>	uint16	—	Total sample count	
<code>preheat</code>	uint16	sample index	Sample index where preheat phase ends	Zero if no pre-heat.
<code>gap</code>	uint16	sample index	Sample index where gap phase ends	Zero if no gap.
<code>main</code>	uint16	sample index	Sample index where main weld phase ends	Typically equals <code>samples</code> .

Important: Phase boundaries in `WAVEFORM_START` are **sample indices** (0-based array positions), not time offsets. Contrast with `WAVEFORM_PHASES` below.

WAVEFORM_DATA

Chunked sample data.

Format: `WAVE-`

`FORM_DATA,timestamp_us,voltage_volts,current_amps,timestamp_us,voltage_volts,current_amps,...`

- Positional CSV: triplets of `(timestamp_us, voltage_volts, current_amps)` .
- `timestamp_us` : Microseconds since weld pulse start.
- Sent in chunks to respect UART/TCP line-length limits.

WAVEFORM_END

Signals the end of the waveform burst.

Format: `WAVEFORM_END`

WAVEFORM_PHASES

Phase timing markers (sent after `WAVEFORM_END`).

Format: `WAVE-`

`FORM_PHASES,preheat_start=T1,preheat_end=T2,gap_start=T3,gap_end=T4,main_start=T5,main_end=T6`

Field	Type	Units	Description
preheat_start	uint32	microseconds	Preheat phase start offset
preheat_end	uint32	microseconds	Preheat phase end offset
gap_start	uint32	microseconds	Gap phase start offset
gap_end	uint32	microseconds	Gap phase end offset
main_start	uint32	microseconds	Main weld phase start offset
main_end	uint32	microseconds	Main weld phase end offset

Important: Phase boundaries in `WAVEFORM_PHASES` are **microsecond time offsets** relative to the weld capture start, not sample indices. This is in contrast to `WAVEFORM_START` boundaries.

ESP32-P4 handling: The P4 allocates an 8 KB buffer (`BUF_SIZE = 8192` , `welder_main.cpp:57`) to accommodate large `WAVEFORM_DATA` lines and forwards all `WAVEFORM_*` packets **raw** to the Flask TCP client via `wifi_bridge_broadcast()` without parsing or storing the samples locally (`welder_main.cpp:358-361`).

Legacy single-line format: The original `WAVEFORM,timestamp,voltage,current,...` (all samples in one line) is now **dead code** in firmware. Flask retains a `_parse_waveform` fallback handler (`app.py:1229-1237`), but it is ignored if chunked waveform assembly is active.

DISPLAY — Smoothed Voltages for UI Labels

Producer: ESP32-P4 (current firmware) and ESP32_8048S043C (legacy firmware)

Consumer: Flask

Frequency: 1 Hz

Format: `DISPLAY,vpack=V1,vcap=V2,cell1=V3,cell2=V4,cell3=V5`

Field	Type	Units	Description
<code>vpack</code>	float	volts	Smoothed pack voltage
<code>vcap</code>	float	volts	Smoothed capacitor voltage
<code>cell1</code>	float	volts	Smoothed cell 1 voltage
<code>cell2</code>	float	volts	Smoothed cell 2 voltage
<code>cell3</code>	float	volts	Smoothed cell 3 voltage

Purpose: Provides low-frequency smoothed/averaged voltages specifically for dashboard UI labels, reducing update noise compared to the high-frequency `STATUS` and `STATUS2` packets.

Current status: Actively used. ESP32-P4 emits `DISPLAY` at 1 Hz after receiving `STATUS2` data (`welder_main.cpp:440-455`).

CELLS — Cell Voltage Request/Response (Legacy)

Producer: ESP32_8048S043C **only** (legacy firmware)

Consumer: Flask (legacy handler)

Frequency: On-demand (in response to `CELLS` command from Flask)

Format: `CELLS,C1=V1,C2=V2,C3=V3`

Field	Type	Units	Description
<code>C1</code>	float	volts	Cell 1 voltage
<code>C2</code>	float	volts	Cell 2 voltage
<code>C3</code>	float	volts	Cell 3 voltage

Legacy status: Only the old-board firmware (`ESP32_8048S043C/src/main.cpp:1876-1882`) produces this packet when it receives a `CELLS` command from Flask. The current ESP32-P4 firmware does **not** produce `CELLS` packets, and the STM32 **ignores** the `CELLS` command. Flask retains the `CELLS` request and parser for backward compatibility with old hardware (`app.py:1037, 1159-1161`).

Superseded by: `DISPLAY` packets (which provide similar smoothed voltage data at 1 Hz) and continuous `STATUS2` telemetry.

Non-Packet Notes

WAVEFORM_SAMPLE (Not a Protocol Packet)

`WAVEFORM_SAMPLE_INTERVAL_US` is a **firmware timing constant** in `STM32G474CE/src/main.c:362`, defining the ADC sampling period ($50\ \mu\text{s} = 20\ \text{kHz}$). It is **not** a protocol packet type and should never be documented as one.

Design History and Legacy Notes

- **Redundant fields:** Several fields exist in duplicate for backward compatibility with older Flask versions:
 - `STATUS.joule_target` is an alias of `joule_target_j`.
 - `WELD_DONE.energy_j` is an alias of `energy_weld_j`.
 - Flask parsers handle both variants; newer code prefers the `_j` suffixed versions.
 - **Cell count evolution:** The system was designed for 3-cell LiFePO4 packs. Flask scaffolding for `cell4` exists but is unused (no runtime error; silent no-op).
 - **Waveform format migration:** The protocol migrated from a single-line `WAVEFORM` packet (all samples in one line) to the chunked `WAVEFORM_START / DATA / END / PHASES` stream to handle UART line-length limits at high sample rates. Flask retains the old parser as a fallback, but current firmware does not emit the old format.
 - **ESP32-P4 as a transparent bridge:** The P4 acts as a **relay** for most STM32 telemetry (`STATUS2`, `WAVEFORM_`, `WELD_DONE`), forwarding packets raw without interpretation. It enriches only the `STATUS` packet with WiFi/system/energy fields and produces* the `DISPLAY` packet independently.
-

Breaking Change Policy

Any modification to:

- Field names
- Field order
- Field types
- Units
- Packet structure

requires **coordinated updates** to all four codebases:

1. `STM32G474CE/src/main.c` (emitter)
2. `ESP32P4/main/welder_main.cpp` and `wifi_bridge.cpp` (relay + enrichment)
3. `ESP32_8048S043C/src/main.cpp` (legacy firmware, if still supported)
4. `Spot-Welder-Server/app.py` (Flask parser)

Recommended approach: Prefer **additive changes** (append new fields to the end of packets) over reordering or renaming existing fields, to maintain backward compatibility during transitions.

Document revision: Initial version based on firmware audit (July 2026)